

FUNCTIONAL REQUIREMENTS DOCUMENT (FRD)

Phase 1: Multi-Tenant Base Architecture, Comprehensive Screen Specifications, & User Stories Mapping

System Blueprint Specification Sheet

Designed for Software Engineering, Database Administration, and UI/UX Development Teams
Comprehensive Execution Guide: Translating Spreadsheet Operations into Type-Safe Relational Software Screens

DOCUMENT CONTROL METADATA:

Project Framework: Custom AI-Native Revenue OS (CRM)	Development Phase: Phase 1 MVP Core Baseline Implementation
Target Timeline: First 60 Calendar Days Execution	Database Engine: Supabase PostgreSQL (Fully Federated & Custom Field Elastic)
API Layer Standard: NestJS Enterprise Micro-modules with Prisma ORM	Frontend Interface: Next.js 15 (App Router, TypeScript, shadcn/ui, TanStack Table)

I. PRODUCT OVERVIEW & CORE VISION

1.1 Objective Statement

The primary goal of this system is to fully transition manual sales tracking workflows out of fragmented Excel spreadsheets, paper notebooks, and isolated calendars into a unified application [cite: 5]. Phase 1 provides the structural foundation required to run a high-velocity, multi-tenant sales pipeline [cite: 5]. It establishes clean relational data boundaries, eliminates manual record duplication, enforces time-bound follow-up accountability via system alerts, and creates an elastic schema foundation capable of supporting advanced background AI pipelines in subsequent phases [cite: 5].

1.2 User Personas & Authorization Matrix

The platform natively supports strict multi-tenancy [cite: 5]. This guarantees that multiple independent organizations can register and utilize the software safely, while Row-Level Security (RLS) ensures that Company A can never view, alter, or access the data streams of Company B [cite: 5]. Within each company organization, users are bound to specific functional roles defined below [cite: 5]:

- Super Admin:** Platform owners with universal infrastructure visibility [cite: 5]. They manage master subscription configurations, system-wide parameters, and cross-tenant maintenance blocks [cite: 5].
- Organization Admin (Company Owner):** Possesses complete data control inside their specific tenant [cite: 5]. They manage team permissions, customize pipeline stages, define custom database attributes, and configure third-party application hooks [cite: 5].
- Sales Manager:** Monitored dashboard access to oversee pipeline movement [cite: 5]. They evaluate team performance metrics, handle deal assignments, override sales stages, and generate executive pipeline reports [cite: 5].

- **Sales Representative:** The primary front-line execution role [cite: 5]. They view and manage self-assigned or routed leads, execute pipeline stages, create client notes, log interaction activities, and complete task checklists [cite: 5]. They cannot view records owned by other representatives unless granted permission [cite: 5].
- **Viewer:** Read-only operational visibility [cite: 5]. Used by auxiliary staff or legal stakeholders to verify information states without the ability to modify fields or execute workflows [cite: 5].

II. CORE TECHNOLOGY STACK ARCHITECTURE

To ensure high-speed performance, clean data modeling, and seamless support for future AI modules, the system uses a fully typed and unified technology stack [cite: 5]:

ARCHITECTURAL LAYER	SELECTED TOOL / TECHNOLOGY	STRATEGIC TECHNICAL RATIONALE
Frontend Engine	Next.js 15 (App Router) & TypeScript	Provides fast Server-Side Rendering (SSR) for landing pages and Client-Side rendering for dynamic dashboard widgets [cite: 5]. Full type-safety across views [cite: 5].
UI Base System	shadcn/ui & Tailwind CSS	Ensures a clean, professional, un-styled component baseline allowing full style flexibility without bloat [cite: 5].
Data Table Engine	TanStack Table (v8)	Handles high-performance data rows natively, enabling client-side sorting, column ordering, and multi-selection grouping out-of-the-box [cite: 5].
Drag & Drop Logic	@dnd-kit/core	Provides smooth, high-frame-rate drag-and-drop mechanics for shifting deal cards across Kanban columns [cite: 5].
Backend Framework	NestJS (Enterprise Architecture Pattern)	Enforces a strict dependency injection pattern, splitting modules cleanly (e.g., LeadModule, CustomFieldModule) for high maintainability [cite: 5].
Database & Infrastructure	Supabase PostgreSQL Engine	Combines relational data integrity, native user authentication APIs, cloud storage, and robust Row-Level Security (RLS) under a single platform [cite: 5].
ORM Layer	Prisma ORM	Generates autocompleted, type-safe data queries and migrations directly from the core database schema [cite: 5].
Caching & Async Queues	Redis Cloud & BullMQ	Manages heavy asynchronous background execution lines, such as system alert notifications and status calculation caches [cite: 5].
Communication Nodes	Resend API Engine	A reliable, developer-friendly transactional email node used for security confirmations, alerts, and system notifications [cite: 5].

ARCHITECTURAL LAYER	SELECTED TOOL / TECHNOLOGY	STRATEGIC TECHNICAL RATIONALE
Charts & Visualization	Recharts Library	Renders fluid, SVG-rendered reporting assets and pipelines that adapt perfectly across fluid dashboard windows [cite: 5].

III. PHASE 1 DATABASE ARCHITECTURE & PHYSICAL SCHEMAS

The following design maps out the physical PostgreSQL relational architecture [cite: 5]. It enforces strict data constraints, cascade operations, and indexing strategies to keep page response speeds under 500ms even across millions of rows [cite: 5].

3.1 Primary Schema Definitions (Prisma Schema Model Format)

```
// 1. Tenant Control Object
model Organization {
  id          String    @id @default(uuid())
  name        String
  slug        String    @unique
  industry    String?
  website     String?
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  users       User[]
  leads       Lead[]
  deals       Deal[]
  tasks       Task[]
  notes       Note[]
  activities  Activity[]
  customFields CustomFieldDefinition[]

  @@map("organizations")
}

// 2. Identity & Access Layer
enum UserRole {
  SUPER_ADMIN
  ORG_ADMIN
  SALES_MANAGER
  SALES_REP
  VIEWER
}

model User {
  id          String    @id @default(uuid())
  organizationId String    @map("organization_id")
  email       String    @unique
  fullName    String    @map("full_name")
  role        UserRole   @default(SALES_REP)
  phone       String?
  status      String     @default("ACTIVE")
  createdAt   DateTime   @default(now()) @map("created_at")

  organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
  ownedLeads    Lead[]      @relation("LeadOwner")
  ownedDeals    Deal[]      @relation("DealOwner")

  @@index([organizationId])
  @@map("users")
}
```

```
// 3. Core Lead Ingestion Record
enum LeadStatus {
    NEW
    CONTACTED
    INTERESTED
    QUALIFIED
    REJECTED
}

model Lead {
    id                String          @id @default(uuid())
    organizationId    String          @map("organization_id")
    firstName         String          @map("first_name")
    lastName          String          @map("last_name")
    email             String?
    phone             String?
    company           String?
    source            String          @default("DIRECT")
    status            LeadStatus      @default(NEW)
    ownerId           String?         @map("owner_id")
    createdAt         DateTime        @default(now()) @map("created_at")
    updatedAt         DateTime        @updatedAt @map("updated_at")

    organization      Organization    @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
    owner             User?           @relation("LeadOwner", fields: [ownerId], references: [id],
onDelete: SetNull)
    prospectRecord    Prospect?
    deals             Deal[]
    customValues      CustomFieldValue[]

    @@index([organizationId])
    @@index([ownerId])
    @@index([status])
    @@map("leads")
}

// 4. Prospect Qualification Framework (BANT Standard Elements)
model Prospect {
    id                String          @id @default(uuid())
    leadId            String          @unique @map("lead_id")
    budget            Decimal?        @db.Decimal(15, 2)
    authority          Boolean         @default(false)
    need              String?         @db.Text
    timeline          String?
    qualifiedBy       String          @map("qualified_by")
    qualifiedAt       DateTime        @default(now()) @map("qualified_at")

    lead              Lead            @relation(fields: [leadId], references: [id], onDelete: Cascade)
    deals             Deal[]

    @@map("prospects")
}
```

```
// 5. Sales Deal Module
enum DealStage {
  NEW
  PROPOSAL
  NEGOTIATION
  CONTRACT
  WON
  LOST
}

model Deal {
  id                String      @id @default(uuid())
  organizationId    String      @map("organization_id")
  leadId            String      @map("lead_id")
  prospectId        String?     @map("prospect_id")
  title             String
  value             Decimal      @db.Decimal(15, 2)
  stage             DealStage    @default(NEW)
  probability        Int         @default(10)
  ownerId           String?      @map("owner_id")
  expectedCloseDate DateTime?    @map("expected_close_date")
  createdAt          DateTime     @default(now()) @map("created_at")
  updatedAt          DateTime     @updatedAt @map("updated_at")

  organization      Organization @relation(fields: [organizationId], references: [id],
onDelete: Cascade)
  lead              Lead         @relation(fields: [leadId], references: [id], onDelete:
Cascade)
  prospect           Prospect?   @relation(fields: [prospectId], references: [id], onDelete:
SetNull)
  owner             User?        @relation("DealOwner", fields: [ownerId], references: [id],
onDelete: SetNull)
  contracts          Contract[]

  @@index([organizationId])
  @@index([stage])
  @@map("deals")
}
```

```
// 6. Custom Field Engine Schemas (EAV Pattern)
enum FieldType {
  TEXT
  NUMBER
  DATE
  BOOLEAN
}

model CustomFieldDefinition {
  id                String                @id @default(uuid())
  organizationId    String                @map("organization_id")
  fieldName         String                @map("field_name")
  fieldLabel       String                @map("field_label")
  fieldType         FieldType             @default(TEXT)
  relatedModule    String                @map("related_module")
  createdAt         DateTime              @default(now()) @map("created_at")

  organization      Organization          @relation(fields: [organizationId], references: [id],
onDelete: Cascade)
  values            CustomFieldValue[]

  @@unique([organizationId, fieldName, relatedModule])
  @@map("custom_field_definitions")
}

model CustomFieldValue {
  id                String                @id @default(uuid())
  definitionId      String                @map("definition_id")
  leadId           String                @map("lead_id")
  value            String                @db.Text

  definition        CustomFieldDefinition @relation(fields: [definitionId], references: [id],
onDelete: Cascade)
  lead              Lead                  @relation(fields: [leadId], references: [id], onDelete:
Cascade)

  @@unique([definitionId, leadId])
  @@map("custom_field_values")
}
```

IV. SCREEN SPECIFICATIONS & USER STORIES

This section details the functional blueprint for every screen, including explicit User Stories mapping exactly what operational steps are executed by the user.

4.1 Public & Authentication Screen Layouts

1. PUBLIC LANDING PAGE ROUTE: (^)

Purpose: Explain platform capabilities, display clear pricing matrices, and secure self-service free trial sign-ups [cite: 5].

Core UI Components: Hero layout with active clear CTAs, comparative features matrix, tiered pricing tables, and dynamic FAQs accordion blocks [cite: 5].

Available Actions: [Login] , [Register Workspace] , [Submit Demo Request] [cite: 5].

User Story: "As a prospective buyer (business owner), I want to review an explanatory landing page and pricing table so that I can instantly evaluate product offerings and securely register a 14-day free trial organization instance for my company without friction." [cite: 5]

2. SECURE PORTAL SIGN-IN ROUTE: (^/AUTH/LOGIN^)

Purpose: Authenticate active corporate system users into their tenant dashboard [cite: 5].

Core UI Components: Clean centered form card containing credentials fields, secure OAuth single-sign-on controls, and OTP fallback buttons [cite: 5].

Available Actions: [Authenticate Session] , [Trigger OAuth SSO] , [Dispatch OTP Code] [cite: 5].

User Story: "As an active sales agent, I want to securely sign into the CRM platform using my single-sign-on credentials or a secure OTP confirmation so I can immediately open my daily pipeline dashboard workspace." [cite: 5]

3. ORGANIZATION TENANT WORKSPACE PROVISION ROUTE: (^/AUTH/REGISTER^)

Purpose: Initialize isolated organization rows and administrator accounts [cite: 5].

Core UI Components: Multi-step onboarding state view tracking user details, organizational metadata (company size, vertical vertical), and generation triggers [cite: 5].

Available Actions: [Validate Identity Data] , [Provision Workspace Tenant] [cite: 5].

User Story: "As a company administrator, I want to execute a structured registration flow to initialize my organizational workspace and establish default data settings so my sales team can securely use the platform." [cite: 5]

4.2 Authenticated Workspace Application Screens

4. CENTRAL REVENUE OPERATIONS DASHBOARD ROUTE: (`/DASHBOARD`)

Purpose: Act as the master analytical command hub displaying aggregated pipeline health metrics and velocity metrics [cite: 5].

Core UI Components: Financial metric tiles (Closed Revenue, Forecast Forecast), Lead Count logs, Funnel progression charts, activity loops, and Sales Rep Ranking leaderboard lists [cite: 5].

Available Actions: [Date Filter Toggle], [Refresh Analytical Widgets], [Drill-down to Target Report] [cite: 5].

User Story: "As a sales manager, I want to review aggregated revenue widgets, active pipelines, and team rankings on a consolidated dashboard screen so I can quickly identify high-performing assets and operational drop-offs." [cite: 5]

5. LEADS INTAKE MANAGEMENT HUB ROUTE: (`/DASHBOARD/LEADS`)

Purpose: Organize and process inbound lead listings while supporting rapid multi-view layout changes [cite: 5].

Core UI Components: TanStack-powered sortable rows, responsive dnd-kit Kanban layout columns (New, Contacted, Interested, Qualified), bulk select operation actions [cite: 5].

Available Actions: [Toggle Layout Mode], [Execute Bulk Reassignment], [Create Lead Record Manually] [cite: 5].

User Story: "As a front-line sales representative, I want to toggle between an advanced data table and a drag-and-drop Kanban matrix of my leads so I can organize my daily outreach steps and update record states visually without manual data updates." [cite: 5]

6. LEAD CONTEXT PROFILE VIEW ROUTE: (`/DASHBOARD/LEADS/[ID]`)

Purpose: Provide full contextual history of an isolated account inside a unified split workspace layout [cite: 5].

Core UI Components: Left static identity field definitions (with custom field components); Right interactive tracking workspace with tabs for Overview, Timeline log, Task checkbox trackers, and Document attachments [cite: 5].

Available Actions: [Append Text Note], [Schedule Checklist Task], [Update Custom Attributes], [Convert Lead to Prospect] [cite: 5].

User Story: "As a sales rep, I want to access a split-layout detail view for a lead so I can view their identity parameters on the left, log meeting text notes, review automated timeline events, and upload reference attachments on the right." [cite: 5]

7. QUALIFIED CONTACTS DIRECTORY ROUTE: (`/DASHBOARD/CONTACTS`)

Purpose: List and segment validated individual stakeholders and buyer networks [cite: 5].

Core UI Components: Advanced database tables supporting sorting controls, company name group headers, geographic and vertical filtering controls [cite: 5].

Available Actions: [Apply Industry Filter] , [Export Contact Ledger] , [Cross-link Open Opportunity] [cite: 5].

User Story: "As a sales rep, I want to search and open a structured contact profile so I can examine their historical organizational associations, linked active opportunities, and historical task sequences prior to initiating account phone touches." [cite: 5]

8. PROSPECT BANT QUALIFICATION GATE ROUTE: (`/DASHBOARD/PROSPECTS`)

Purpose: Enforce structured validation frameworks before authorizing lead advancement to an open deal [cite: 5].

Core UI Components: BANT Evaluation forms checking Budget numerical bounds, Authority flags, Need open texts, and Timeline windows [cite: 5].

Available Actions: [Submit Qualification Parameters] , [Approve and Escalate to Opportunity] [cite: 5].

User Story: "As a sales representative, I want to execute a mandatory BANT qualification form for a lead so that the platform validates budget parameters and decision authority before approving escalation into an active sales deal." [cite: 5]

9. VISUAL DEAL PIPELINE GRID ROUTE: (`/DASHBOARD/DEALS`)

Purpose: Manage high-value transactional negotiations and track weighted forecasts [cite: 5].

Core UI Components: Multi-column drag-and-drop board (New, Proposal, Negotiation, Contract, Won, Lost), forecast analytics toggles displaying probability adjustments [cite: 5].

Available Actions: [Shift Deal Stage] , [Toggle Forecast View] , [Log Competitor Infractions] [cite: 5].

User Story: "As a sales rep, I want to track active deals on a visual sales board and review contract closure probabilities so I can maintain pipeline momentum and focus my energy on high-value transactions." [cite: 5]

10. CONTRACT MANAGEMENT LEDGER ROUTE: (`/DASHBOARD/CONTRACTS`)

Purpose: Control contract drafting, sending, and electronic execution states [cite: 5].

Core UI Components: Status log list components (Draft, Sent, Signed, Expired), web-safe PDF canvas viewer panels, and secure e-signature validation logs [cite: 5].

Available Actions: [Upload Agreement PDF] , [Disseminate Signature Links] , [Enforce Version Baseline] [cite: 5].

User Story: "As a sales operations lead, I want to access a contract tracking screen and verify real-time signature states so that I can seamlessly monitor document progression and track execution milestones." [cite: 5]

11. ACTIVE CUSTOMER LEDGER ROUTE: (`/DASHBOARD/CUSTOMERS`)

Purpose: Retain profiles of paying entities to maintain customer success touches and unlock upsell pathways [cite: 5].

Core UI Components: Customer directory lists tracking life-time value summaries, cross-linked executed agreements, and customer success timers [cite: 5].

Available Actions: [Segment Retention Cohort] , [Open Post-Sale Incident Log] [cite: 5].

User Story: "As a customer success associate, I want to filter our directory of verified active accounts so I can log recurring support touches and identify accounts ripe for expansion packages." [cite: 5]

12. TASKS CHECKLIST & CALENDARING ROUTE: (`/DASHBOARD/TASKS`)

Purpose: Consolidate multi-module actions into structured schedules to prevent lead drops [cite: 5].

Core UI Components: Multi-mode calendar views, action filter checkboxes (Call, Meeting, Follow-up), priority flags (Low, Med, High) [cite: 5].

Available Actions: [Check Complete Task] , [Reschedule Deadline] , [Filter Task Categories] [cite: 5].

User Story: "As a sales rep, I want to check my task engine across a calendar grid view so I can execute high-priority calls and clear out overdue communication follow-ups systematically." [cite: 5]

13. CUSTOMIZATION & ADMINISTRATIVE RULES SETTINGS ROUTE: (/DASHBOARD/SETTINGS)

Purpose: Allow administrators to customize schemas, fields, and pipelines dynamically without code updates [cite: 5].

Core UI Components: Field Definition builders (Label text, variable type selectors), pipeline column management lists, custom filtering configuration logs [cite: 5].

Available Actions: [Define Custom Variable Mapping], [Alter Pipeline Sequences], [Save Global View Shortcut] [cite: 5].

User Story: "As a tenant administrator, I want to add a unique field placeholder like 'GST Number' via the control panel without writing code, so that our agents can log regional tax parameters directly onto incoming enterprise profiles." [cite: 5]

V. STRATEGIC LONG-TERM PRODUCT ARCHITECTURE (ALL PHASES)

To retain architectural coherence across subsequent engineering intervals, development follows four sequential phases [cite: 5]:

Phase 1: Foundation Baseline (Current Specification — Months 1-2): Core multi-tenant architecture setup, relational data schemas, views, tasks, analytics metrics, dynamic custom field definition engines, and view preservation frameworks [cite: 5].

Phase 2: Omni-channel Automation Gateways (Months 3-4): Integration of Meta’s WhatsApp Business API, background IMAP email sync tracking, business call logging engines, custom pricing quotation builders, and automated messaging drip sequences [cite: 5].

Phase 3: Contextual AI Intelligence Layer (Months 5-6): Introduction of predictive lead scoring, automated speech-to-text call transcription, contact record auto-enrichment scripts, and automated proposal engagement tracking [cite: 5].

Phase 4: Autonomous Sales Operating Optimization (Months 7+): Integration of the **AI Next Best Action Command Engine**, live call objection flash assistance systems, autonomous WhatsApp conversational agents, and predictive cash flow reporting [cite: 5].

--- END OF PRODUCTION TECHNICAL FUNCTIONAL REQUIREMENTS DOCUMENT ---
This blueprint represents the formal operational master copy for engineering deployment [cite: 5].
Authorized for production database indexing, application service provisioning, and core repository initiation [cite: 5].